

HySELoc: a hybrid semantic-enhanced approach for interpretable vulnerability localization

Xiansheng Cao¹, Junfeng Wang^{2,*}, Peng Wu³

¹School of Cyber Science and Engineering, Sichuan University, No. 2, Section 2, Chuanda Road, Shuangliu District, Chengdu 610207, China

²College of Computer Science, Sichuan University, No. 24, Section 2, South First Ring Road, Wuhou District, Chengdu 610065, China

³School of Information and Engineering, Sichuan Tourism University, No. 459, Hongling Road, Longquanyi District, Chengdu 610100, China

*Corresponding author. College of Computer Science, Sichuan University, Chengdu 610065, China. E-mail: wangjf@scu.edu.cn

Abstract

Early detection and localization of code vulnerabilities are crucial for software security. However, existing interpretable localization methods rely on attention over code sequences, failing to capture syntactic structures and semantic dependencies, limiting accuracy. Additionally, abstract syntax trees (ASTs), a widely used syntactic representation, suffer from node redundancy and alignment issues, while existing graph-based representations are overly complex and lack fine-grained semantic modeling, making them ineffective for interpretable vulnerability localization. To address these challenges, this paper proposes HySELoc, a hybrid semantic-enhanced framework that models context, syntax, and semantics across multiple levels by integrating code sequences, AST subtree sequences, and a tokenized hybrid semantic graph (THSG). AST subtree sequences segment ASTs at the statement level, while THSG, built at the token level, defines six semantic edge types to enhance interpretable localization. A hierarchical attention mechanism computes token-level contributions, adaptively weighting and fusing them for precise line-level localization. Experiments show HySELoc outperforms baseline methods in F1-score for vulnerability detection on REVEAL, D2A, and Big-Vul, with average absolute improvements in top-10 accuracy of 23.4 and 15.75 percentage points on D2A and Big-Vul, respectively. Additionally, it successfully detected two 0-day vulnerabilities (CVE-2024-6062/6064), validating its practical value.

1. INTRODUCTION

Code vulnerabilities refer to errors, defects, or weaknesses in source code [1], which can lead to system crashes or expose software to attacks, posing significant security risks [2]. For example, the Log4Shell vulnerability (CVE-2021-44228) [3] in the Apache Log4j library allows attackers to execute arbitrary code through malicious input, compromising millions of servers worldwide [4]. Another notable case is the Heartbleed vulnerability (CVE-2014-0160) [5], caused by a memory handling flaw in the OpenSSL library, enabling attackers to extract sensitive information from server memory, affecting over 500 000 systems and resulting in billions of dollars in damages [6]. These high-impact vulnerabilities underscore the fragility of the software supply chain and highlight the critical need for early detection and localization. However, traditional static analysis tools, such as Rats [7], Flawfinder [8], CppCheck [9], and Checkmarx [10], primarily rely on manual auditing, making them inadequate for handling the scale and complexity of modern codebases.

Deep learning-based vulnerability detection has become a key approach for identifying and localizing code vulnerabilities. Its core lies in transforming code into intermediate representations and leveraging neural networks to learn vulnerability patterns [11–16]. However, significant challenges remain in vulnerability localization. Existing techniques can be broadly categorized into two approaches: (1) binary classification of code fragments or individual statements to determine vulnerability presence

[17–19], and (2) attribution-based methods that analyze a model's internal mechanisms to compute statement-level contribution scores for localization [20]. While the latter has gained attention for its interpretability, its effectiveness is limited by single-code representations, failing to incorporate hierarchical syntactic structures and critical semantic dependencies, which restricts localization accuracy. Currently, a major challenge in interpretability analysis is the lack of a unified modeling approach that captures both the hierarchical syntax and deep semantic relationships within code. Addressing this dual technical bottleneck is crucial for advancing interpretable vulnerability localization.

An important challenge is how to effectively analyze code syntax for localization. While abstract syntax trees (ASTs) capture hierarchical syntax, their node count grows exponentially with code complexity. Many nodes serve only parsing purposes, diluting key features and increasing localization difficulty. More critically, ASTs represent syntax units as nodes but lack direct alignment with source code lines, causing granularity mismatch. Existing methods fail to aggregate these discrete node features into specific lines through simple strategies. Therefore, a representation is needed that both preserves hierarchical syntax and aligns with line numbers to enable effective syntax-based vulnerability localization.

Another key challenge is effectively analyzing code semantics for localization. Existing methods primarily rely on code property

Received: March 16, 2025. Revised: June 6, 2025. Accepted: July 7, 2025

© The British Computer Society 2025. All rights reserved. For commercial re-use, please contact reprints@oup.com for reprints and translation rights for reprints. All other permissions can be obtained through our RightsLink service via the Permissions link on the article page on our site—for further information please contact journals.permissions@oup.com.

graphs (CPGs) and their simplified variant, program dependence graphs (PDGs), to extract semantic information. CPGs extend ASTs by incorporating data flow and control flow, providing richer semantics. However, their structural complexity and redundancy make them impractical for interpretable vulnerability localization. While PDGs are widely used in binary classification-based vulnerability localization [18], they have limitations in interpretability. Since PDGs represent basic blocks as nodes, they struggle to capture token-level fine-grained semantics, such as variable names and operators, which restricts their effectiveness in line-level vulnerability localization. Thus, there is a critical need for a more effective semantic representation that enhances interpretability and adaptability in vulnerability localization.

To address these challenges, this paper introduces HySELoc, an interpretable vulnerability localization framework with hybrid semantic enhancement, achieving key advancements in multiple areas. First, in hierarchical syntax modeling and line-level alignment, HySELoc segments ASTs into statement-level subtrees, preserving syntactic structure while reducing redundancy. This allows precise mapping of syntactic features to specific code lines, enhancing interpretability in vulnerability localization. Second, in lightweight semantic graph representation, HySELoc introduces tokenized hybrid semantic graph (THSG), a novel graph-based representation that models tokens as nodes and incorporates six types of semantic edges. This approach reduces graph complexity while effectively capturing data flow, control flow, and variable dependencies, ensuring token-level graph representations enhance interpretability in vulnerability localization. Finally, in model fusion, HySELoc employs a joint Transformer-GNN framework, utilizing a hierarchical attention mechanism to extract fine-grained features from code sequences, AST subtree sequences, and THSG. Through an adaptive weighting strategy, it aggregates token-level attention scores into line-level contributions, enhancing both the interpretability and accuracy of vulnerability localization.

Experiments demonstrate that HySELoc achieves significant improvements over baseline methods in function-level detection, with F1-scores notably higher across three benchmark datasets (REVEAL, D2A, and Big-Vul). In line-level localization, top-10 accuracy reaches 81% on D2A and 90% on Big-Vul, showing substantial gains compared with baselines. In real-world applications, HySELoc successfully identified two 0-day vulnerabilities (CVE-2024-6062/6064) in the Gpac project, validating its practical engineering applicability.

The contribution of this paper can be summarized as follows:

- **Hierarchical Hybrid Semantic-Enhanced Framework:** integrates code sequences, AST subtrees, and a lightweight THSG for multilevel context, syntax, and semantic modeling, addressing the limitations of single-representation approaches in interpretable vulnerability localization.
- **Hierarchical Syntax Modeling and Line-Level Alignment:** introduces statement-level AST subtree segmentation to reduce node redundancy and resolve line-level misalignment, providing fine-grained syntactic support for interpretable vulnerability localization.
- **Token-Level Semantic Graph:** introduces a lightweight THSG that models fine-grained semantic relationships at the token level, providing support for interpretable vulnerability localization.
- **Real-World Vulnerability Verification:** validates HySELoc's effectiveness in real-world applications by dynamically

confirming 0-day vulnerabilities detected by the model, demonstrating its practical applicability.

The rest of the paper is organized as follows. Section 2 reviews related work. Section 3 details the design of HySELoc. Section 4 describes the experimental setup. Section 5 presents the experimental results. Section 6 serves as the discussion section. Finally, Section 7 concludes the paper.

2. RELATED WORK

To review recent advances in vulnerability detection and localization, we introduce related work from two perspectives: code representation learning methods and vulnerability localization approaches.

2.1 Code representation learning methods

Code representation is fundamental to vulnerability detection, with existing approaches primarily focusing on sequences and trees/graphs.

Sequence-based models treat code as a token sequence and leverage neural networks to learn textual semantics. For example, Li et al. [21] generate code gadgets via program slicing and detect vulnerabilities using BiLSTM. Li et al. [22] further represent code as vulnerability candidates incorporating syntax and semantics and train a BiLSTM model for vulnerability detection. Additionally, Transformer-based models enhance sequence modeling through self-attention mechanisms. Fu et al. [20] train vulnerability patterns using CodeBERT [23], while Zhang et al. [24] propose a method that learns syntax-based execution paths for vulnerability detection. However, these methods flatten the code structure, leading to the loss of syntactic hierarchy and key semantic relationships, making them ineffective in capturing complex vulnerability patterns.

Tree-/graph-based models attempt to extract syntactic and semantic information through structured representations. Zhou et al. [25] encode code as a graph with four edge types and apply gated graph neural network (GGNN) to learn vulnerability features. Chakraborty et al. [26] leverage CPG with GGNN for vulnerability detection, while Li et al. [27] employ PDG and graph convolutional network (GCN) [28] to model vulnerability patterns. Ding et al. [17] combine GGNN and Transformer to enhance semantic understanding for vulnerability identification, while Tian et al. [29] adopt recursive neural network and Transformer for vulnerability detection. Liu et al. [30] explore the integration of language models with online distilled graph neural network (GNN) for vulnerability detection. However, existing tree-/graph-based representations suffer from AST node redundancy, line-granularity misalignment, and the coarse-grained semantic modeling of CPG/PDG, making them insufficient for interpretable vulnerability localization, which requires fine-grained syntax and semantic alignment.

HySELoc addresses these limitations by serializing AST subtrees to reduce redundant nodes and achieve line-level alignment. It introduces THSG, a lightweight graph representation, which models fine-grained semantic relationships at the token level, overcoming the coarse node granularity and redundant semantic edges of traditional CPG/PDG, which limit their applicability to interpretable vulnerability localization. Furthermore, HySELoc employs multimodal joint learning to dynamically integrate syntax and semantic features, overcoming the limitations of single-sequence or graph-based methods that fail to effectively combine syntax and semantics.

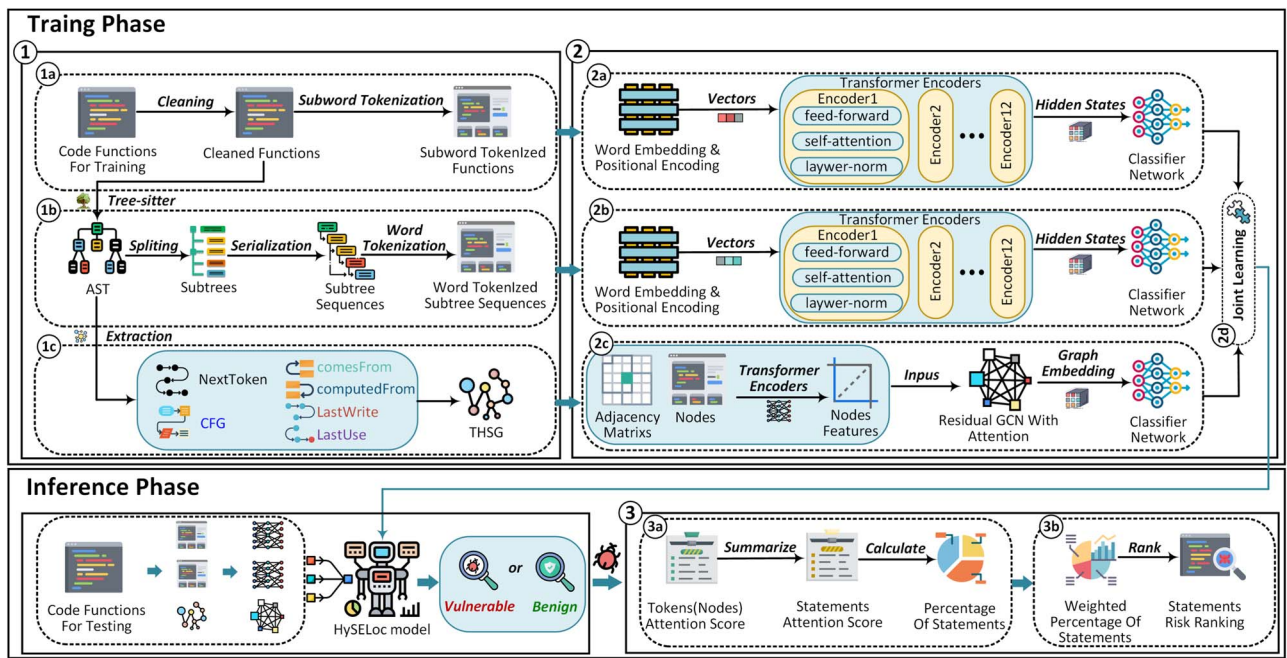


Figure 1. Overall framework of HySELoc.

2.2 Deep learning-based vulnerability localization methods

Deep learning-based vulnerability localization methods can be categorized into classification-based approaches and interpretable methods. The latter can be further divided into model-agnostic and model-specific approaches.

Classification-based methods treat code fragments or statements as binary classification tasks. For example, Li *et al.* [19] extract vulnerability candidates using program slicing and apply BiLSTM to identify vulnerabilities, assigning confidence scores to statements. Hin *et al.* [18] utilize GNNs and Transformers to learn graph representations, enabling vulnerability localization through node classification. Ding *et al.* [17] further enhance classification performance by integrating ensemble learning frameworks and pretraining techniques. Although classification-based methods show promise for vulnerability localization, they essentially function as black-box models, failing to meet the interpretability demands of high-risk vulnerability auditing.

Model-agnostic interpretability methods aim to explain deep learning model decisions, providing insight into prediction results. For instance, GradientSHAP [31] estimates SHAP values by computing gradients combined with random background sampling, while LIG [32] integrates gradients to approximate the marginal contribution of input features. Saliency [33] evaluates feature importance by computing gradients of the output with respect to the input, and DeepLiftSHAP [31] extends DeepLIFT with SHAP values, comparing activation values with a baseline to compute contribution scores. However, these methods require multiple input perturbations, resulting in high computational costs. Moreover, such methods are originally designed for image or natural language tasks and often fail to capture the structural and semantic dependencies inherent in source code. For example, perturbing a token in code may invalidate the syntax or introduce misleading semantics, which affects both model behavior and explanation quality. Therefore, applying these methods directly to code often results in noisy or uninformative attribution maps.

Model-specific interpretability methods leverage internal model information and features to explain function-level predictions. Li *et al.* [27] employ GCNs to learn graph-based

vulnerability representations and use GNNExplainer [34] and masking techniques to achieve subgraph-level interpretability for fine-grained detection. Fu *et al.* [20] analyze subword contributions using Transformer-based self-attention to identify potential vulnerable lines. However, these approaches have notable limitations: GNNExplainer [34] remains at a coarse-grained subgraph level, while Fu *et al.*'s method [20] relies solely on sequence-based representations, failing to effectively integrate syntax and semantics, thereby limiting localization accuracy and interpretability. Specifically, GNNExplainer generates explanations by identifying influential subgraphs. While effective for general graph tasks, it often yields coarse or fragmented regions when applied to program graphs such as ASTs or DFGs, where precise, context-aware attribution is essential. Moreover, its optimization-based masking process is computationally expensive and prone to instability, limiting its practicality for consistent statement-level localization. Similarly, LineVul [20], though effective at identifying vulnerable lines using attention distributions, is built on a purely sequence-based Transformer architecture. This restricts its ability to capture deeper semantic and structural dependencies within code. As a result, it may misattribute importance to superficial tokens or overlook control/data flow elements essential for understanding vulnerabilities.

HySELoc leverages attention-based attribution over the integrated graph, providing line-level contribution scores that are both semantically coherent and structurally grounded. By aligning attribution results with meaningful syntactic and semantic constructs, HySELoc not only improves localization accuracy but also provides explanations that are more intuitive and actionable for developers.

3. THE PROPOSED APPROACH

This section introduces our proposed HySELoc method, with its overall framework shown in Fig. 1. HySELoc follows a two-step approach: it first identifies vulnerable functions and then locates the vulnerable statements. The process consists of two main phases: model training and model validation.

Algorithm 1 AST Subtrees Splitting and Serialization**Require:** AST T , max depth d , max size n **Ensure:** Subtree Sequences S

```

1: function SPLIT( $r, C, d, n, D$ )
2:   if  $r$  is a statement then
3:     if  $r.type \in C$  then
4:       if  $Size(r) < n$  and  $Depth(r) < d$  then
5:          $t \leftarrow ConstructSubtree(r)$ 
6:          $D.append(t)$ 
7:       else
8:          $t \leftarrow ConstructSubtree(r.header)$ 
9:          $D.append(t)$ 
10:      for all  $child \in r.body$  do
11:        SPLIT( $r, C, d, n, D$ )
12:     else
13:        $t \leftarrow ConstructSubtree(r)$ 
14:        $D.append(t)$ 
15:   else
16:     for all  $child \in r.children$  do
17:       SPLIT( $r, C, d, n, D$ )
18: function SERIALIZE( $tree$ )
19:    $seq \leftarrow [(tree.type, tree.code)]$ 
20:    $stack \leftarrow [tree.root]$ 
21:   while  $stack \neq \emptyset$  do
22:      $curr \leftarrow stack.pop()$ 
23:     for all  $child \in Reverse(curr.children)$  do
24:        $stack.push(child)$ 
25:        $seq.append((child.type, child.code))$ 
26:   return  $seq + [ASTSUBTREE]$ 
27:  $r \leftarrow getRootNode(T)$   $\triangleright$  Root node of the AST
28:  $C \leftarrow \{FuncDef, Compound, If, Else, While, Switch, Try, For, Do\}$   $\triangleright$  Compound node types
29:  $D \leftarrow \emptyset$   $\triangleright$  Statement-level subtree collection
30:  $S \leftarrow \emptyset$   $\triangleright$  Subtree sequences collection
31: SPLIT( $r, C, d, n, D$ )
32: for all  $subtree \in D$  do
33:   if  $Size(subtree) > 1$  then
34:      $S.append(Serialize(subtree))$ 
35: return  $S$ 

```

3.1 Identify vulnerability functions

HySELoc's vulnerability function identification is one of its core tasks, consisting of two main steps: code preprocessing and model construction.

3.1.1 Code preprocessing

Code preprocessing corresponds to **Step ①** in Fig. 1, where we prepare code representations for each of the three submodels.

①a Construction of Code Sequences

Code cleaning. We first clean the training code functions by removing comments and blank statements, which do not affect program logic but introduce noise. This step standardizes code style and helps the model focus on core logic, yielding "Cleaned Functions" for downstream processing.

Code sequence tokenization. In this step, we apply the efficient Byte Pair Encoding (BPE) algorithm [35] to perform subword tokenization on the "Cleaned Functions." Specifically, we use the tokenizer from the CodeBERT pre-trained model [23] to ensure good generalization. BPE starts by splitting the input into characters and iteratively merges the most frequent pairs until a predefined

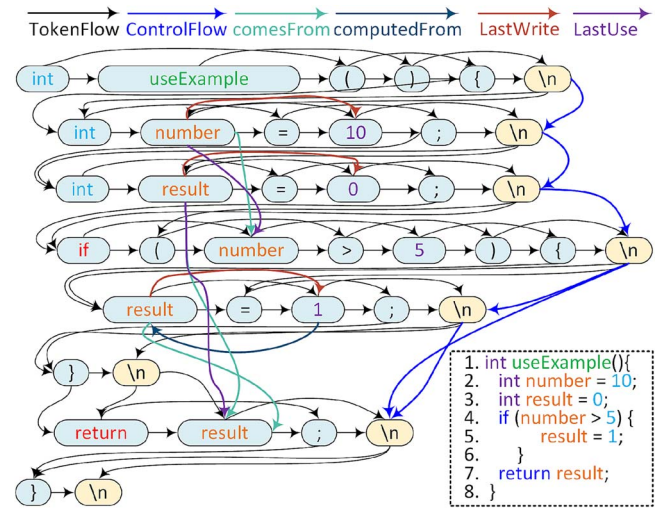


Figure 2. THSG example based on the code in the lower right. It contains 75 TokenFlow edges, six ControlFlow edges, three ComesFrom edges, one ComputedFrom edge, three LastWrite edges, and two LastUse edges. After removing five duplicate edges, this THSG has 85 edges.

vocabulary size is reached, producing the "Subword Tokenized Functions."

①b Construction of Subtree Sequences

Splitting AST into subtrees. We first decompose the AST T into a set of statement-level subtrees to reduce structural complexity. As shown in Algorithm 1, the tree is traversed in preorder starting from the root. For each node, we check whether it is a statement. If its type belongs to the compound set C , we assess the size and depth of its subtree. When both are within thresholds n and d , the subtree is retained; otherwise, only the header is kept, and each body child is recursively processed using the Split function. Non-compound statements are directly retained, while non-statement nodes are recursively expanded. All resulting subtrees are built via ConstructSubtree and collected into D , forming a sequence of independent, statement-level "Subtrees" for subsequent serialization.

Serialization of subtrees. Based on the Serialize function in Algorithm 1, each subtree in D is converted into a linear sequence using depth-first traversal. Starting from the root node, the algorithm visits all descendants in order and records each node as a $\langle type, code \rangle$ pair, which is appended to the sequence seq . A special delimiter "ASTSUBTREE" marks the end of each subtree. Subtrees with only one node are discarded to reduce noise. The resulting set of structured "Subtree Sequences" serves as input for downstream processing.

Word tokenization of subtree sequences. To preserve semantics, we avoid BPE and instead train a word-level tokenizer on the subtree corpus. The subtree sequence list S is then tokenized at the word level, producing a sequence set referred to as "Word Tokenized Subtree Sequences."

①c Construction of THSG

THSG integrates six types of edges: TokenFlow, ControlFlow, ComesFrom, ComputedFrom, LastWrite, and LastUse. Figure 2 shows a THSG constructed for a function in the figure.

TokenFlow Edges. Inspired by [36–38], we design TokenFlow edges to capture sequential relationships between tokens. By linking adjacent tokens in code order (with the number of connections determined empirically), TokenFlow preserves local context and short-range dependencies, thereby enhancing the model's ability to accurately localize vulnerabilities.

ControlFlow edges. Following [24], we construct a syntax-based control flow graph (CFG) on the AST. To form ControlFlow edges, we insert line break “\n” tokens after each statement and treat them as CFG nodes. This structure enhances the model’s understanding of execution logic, facilitating the detection of vulnerability paths and logical errors in conditional branches.

ComesFrom edges and ComputedFrom edges. The ComesFrom edge indicates the origin of a variable’s value, while the ComputedFrom edge signifies that the variable’s value has been derived through computation or manipulation. Inspired by GraphCodeBERT [39], we generate these edges by traversing AST nodes and identifying key elements such as assignments, variable declarations, and control structures. These edges enable variable and data flow tracking, supporting the detection of issues such as uninitialized variables, data leakage, buffer overflows, and improper conditions.

LastWrite edges and LastUse edges. Following [36], we extract LastWrite and LastUse edges by analyzing assignments, declarations, and variable usages in the AST. LastWrite edges mark the most recent assignment to a variable, while LastUse edges indicate its last access. These edges facilitate tracking variable lifecycles, aiding in the detection of vulnerabilities such as race conditions, data leakage.

Compared with traditional representations such as CPGs or PDGs, THSG offers a more lightweight and efficient alternative for capturing semantic dependencies. While CPGs often involve multiple node types (e.g. call expressions, variable declarations, literals) and dense interprocedural edges that increase graph complexity, our THSG focuses solely on token-level semantic links with six well-defined edge types.

In practice, on the datasets used in this study (D2A [40] and BigVul [41]), we achieved a 100% construction success rate for THSG. In contrast, CPG and PDG, which rely on Joern [42], failed to generate graphs for an average of 7.23% of functions. THSG also significantly reduces preprocessing overhead, with graph construction time decreased by approximately 94% compared with CPG and PDG. Moreover, during model training, THSG reduces average GPU memory consumption by 40%. These results demonstrate the practicality and scalability of THSG for large-scale vulnerability localization.

3.1.2 Construction of the HySELoc model

The construction of the HySELoc model, consisting of three submodels, corresponds to Step ② in the overall framework illustrated in Fig. 1.

②a Construction of Code Sequence Submodel

Constructing model inputs. The input to this submodel consists of the “Subword Tokenized Functions” obtained in Step ①a. Each function is encoded as a sequence of subword tokens enriched with positional information. These token sequences are embedded using a pre-trained embedding matrix and positional encodings to produce the final input representation for the Transformer.

12-layer Transformer encoder. We adopt a standard 12-layer Transformer encoder with multi-head self-attention and position-wise feed-forward networks, following the architecture of CodeBERT. This encoder produces deep contextualized representations of the input token sequence.

Classification network. The final hidden states produced by the Transformer encoder are passed through a linear layer with a softmax activation to generate the probability that the input function is vulnerable.

Loss function. To address class imbalance in vulnerability detection, we adopt the Focal Loss [43], which down-weights easy samples and focuses training on hard examples. The loss is defined as

$$FL(\hat{y}, y) = \begin{cases} -\alpha(1 - \hat{y})^\gamma \log(\hat{y}), & \text{if } y = 1 \\ -(1 - \alpha)\hat{y}^\gamma \log(1 - \hat{y}), & \text{if } y = 0 \end{cases}$$

Here, y is the ground truth label, $\hat{y}$ is the predicted probability, α balances class importance, and γ controls the focusing factor. This formulation helps the model learn effectively from minority vulnerable samples.

②b Construction of Subtree Sequences Submodel

②c Construction of THSG Submodel

Constructing residual GCN inputs. The input to this submodel is the THSG generated in Step ①c. We define THSG as $G(V, E)$, where V is the set of token nodes and E is the set of semantic edges. To process THSG using residual GCN, we construct two key inputs:

Adjacency matrix. A matrix $\mathbf{A} \in \mathbb{R}^{k \times k}$ representing edge connectivity in THSG, where k is the number of nodes (i.e. code tokens), and $\mathbf{A}_{pq} = 1$ if there is an edge between nodes v_p and v_q , and $\mathbf{A}_{pq} = 0$ otherwise.

Node feature matrix. Each node in THSG corresponds to a code token. We obtain token embeddings using CodeBERT [23], resulting in a feature matrix $\mathbf{T} \in \mathbb{R}^{k \times d}$, where d is the embedding dimension. These embeddings are integrated with THSG’s structure to capture both sequential and structural token relationships, enhancing the semantic representation of code for vulnerability detection.

Residual GCN with attention layer. We employ a two-layer residual GCN with an attention mechanism to enhance feature aggregation. Each GCN layer updates node features as

$$\mathbf{H}^{(n+1)} = \mathbf{H}^{(n)} + \sigma(\hat{\mathbf{A}}\mathbf{H}^{(n)}\mathbf{W}^{(n)}),$$

where $\hat{\mathbf{A}}$ is the normalized adjacency matrix, $\mathbf{W}^{(n)}$ is a trainable weight matrix, and σ denotes a nonlinear activation function.

Let $\mathbf{H}$ denote the output of the final GCN layer. An attention mechanism is then applied to refine node representations:

$$\alpha = \text{sigmoid}(\mathbf{H}\mathbf{W}_a), \quad \mathbf{H}' = \alpha \odot \text{ReLU}(\mathbf{H}\mathbf{W}_i) \odot \mathbf{M},$$

where $\mathbf{W}_a$ and $\mathbf{W}_i$ are trainable weight matrices, $\mathbf{M}$ is a masking matrix used for feature selection, and $\odot$ denotes element-wise multiplication. Here, $\mathbf{H}'$ represents the final attention-enhanced node features.

Classification network. The final graph representation is passed through the same classification network as in Step ②a to produce the vulnerability probability.

Loss function. The same Focal Loss function described in Step ②a is used to address class imbalance and focus learning on challenging examples.

②d Joint Learning of the Three Submodels

After extracting features from the three submodels, we perform joint learning by combining their losses and classification probabilities via weighted summation, enabling more

comprehensive vulnerability analysis and improved accuracy. The overall loss and predicted probability are defined as

$$\mathcal{L}_{\text{joint}} = \sum_{l=1}^L w_l \mathcal{L}^l, \quad \mathcal{P}_{\text{joint}} = \sum_{l=1}^L w'_l \mathcal{P}^l,$$

where $L = \{\text{Seq}, \text{SubtreesSeq}, \text{THSG}\}$ refers to the code sequences, subtree sequences, and THSG submodels, respectively. The learnable weights w_l and w'_l control each submodel's contribution to the joint loss and prediction, enabling the model to dynamically integrate multiple feature sources.

3.2 Locate vulnerability statements

Corresponding to **Step** ③, in this step, we achieve vulnerability localization based on the intuition that statements that contribute more to the classification of vulnerabilities are more likely to be vulnerable statements. To this end, we explicitly design a contribution-based localization mechanism that aggregates attention-derived scores from three complementary submodels.

③a Collection of the Percentage Contribution of Submodel Statements

For each vulnerability function identified by the model, we compute the percentage contribution of each statement within the function by analyzing the attention scores from different submodels.

Summarizing attention scores. In the code sequence submodel, subword attention scores from all 12 Transformer layers are aggregated and grouped by statement boundaries using line breaks (“\n”). In the subtree sequence submodel, attention scores are aggregated by the “ASTSUBTREE” separator. In the THSG submodel, node-level scores are summed and mapped to statements using line positions, consistent with the code sequence strategy.

Calculating the percentage of statements. For each function, we normalize the aggregated statement-level attention scores across all statements to sum to one. This yields the relative contribution (i.e. percentage) of each statement within its function, making contributions comparable across functions.

③b Ranking and Locating Vulnerable Statements

After normalizing statement-level contributions from each submodel, we compute the overall score via weighted summation:

$$\mathcal{S}_{\text{joint}} = \sum_{l=1}^L w_l \mathcal{S}^l,$$

where $L = \{\text{Seq}, \text{SubtreesSeq}, \text{THSG}\}$ refers to the three submodels, $\mathcal{S}^l$ denotes the contribution score from submodel l , and w_l is a learnable weight controlling its influence.

Ranking and vulnerability localization. We rank all statements in descending order of $\mathcal{S}_{\text{joint}}$. The intuition behind this ranking is that statements with higher contribution scores have a greater impact on the classification decision and are therefore more likely to be vulnerable. By aggregating contributions from all submodels, this method accurately identifies critical vulnerable statements.

4. EXPERIMENTAL DESIGN

4.1 Studied datasets

To evaluate HySELoc, we utilized three widely-used, high-quality datasets: REVEAL [26], D2A [40], and Big-Vul [41]. These datasets are extensively adopted in vulnerability detection research due to their comprehensive coverage of real-world vulnerabilities and

diverse codebases. The basic factual statistics of these datasets, including the number of vulnerable and benign functions, as well as statements, are summarized in Table 1.

4.2 Baselines

(1) Function-level vulnerability detection.

To comprehensively evaluate HySELoc's function-level detection capability, we select six state-of-the-art baselines based on two criteria: representative techniques and reproducibility.

The selected baselines cover diverse code representation paradigms, including sequence-based models (VulDeePecker [21], SySeVR [22]), graph-based models (REVEAL [26], IVDetect [27]), hybrid approaches (VELVET [17], which combines GNNs and Transformer-based models), and Transformer-based methods (LineVul [20]).

All baselines utilize open-source implementations or reproducible experimental setups to ensure fair comparisons. These baselines span key methodologies in vulnerability detection research, including sequence-based, graph-based, hybrid, and Transformer-based approaches, ensuring a rigorous evaluation of HySELoc's hybrid semantic-enhanced framework.

(2) Statement-level vulnerability localization.

For statement-level localization, we adopt two types of baselines: model-specific interpretable methods and model-agnostic interpretability techniques.

The first type includes LineVul [20], a state-of-the-art Transformer-based approach that leverages self-attention mechanisms for line-level vulnerability localization. The second type comprises four widely-used, model-agnostic interpretability methods: GradientSHAP [31], LIG [32], Saliency [33], and DeepLiftSHAP [31]. These methods are classic and representative approaches for feature attribution and contribution analysis in deep learning models.

This dual-baseline strategy ensures a comprehensive evaluation: LineVul represents the state-of-the-art in model-specific interpretable vulnerability localization, while the model-agnostic methods provide a broader perspective on explainability, enabling a thorough assessment of HySELoc's localization capabilities.

4.3 Experimental setup

We implemented HySELoc entirely in Python, primarily utilizing the PyTorch [44] and Transformers [45] libraries. For AST generation, we employed the widely used Tree-sitter parser [46]. The dataset was split into training, validation, and test sets in an 8:1:1 ratio.

For the code sequence and subtree sequence submodels, we adopted the same configuration as CodeBERT [23], using 12 Transformer encoder layers, 12 attention heads, and a maximum token length of 512. The THSG submodel incorporates a two-layer residual GCN, encoding graph nodes into 768-dimensional vectors with a maximum of 512 nodes. The CodeBERT pretrained model was used for node embedding, and TokenFlow connected three successive neighboring nodes.

For training, we set the learning rate to 2e-5 with a linear decay strategy, used a batch size of 64, and trained the model for 30 epochs with the Adam optimizer. All experiments were conducted on a Linux server equipped with two Nvidia A800-80G GPUs and a 48-core 3.20GHz Intel Xeon Gold 6146 CPU.

4.4 Evaluation metrics

Function-level vulnerability detection. Following the standard practices in deep learning-based vulnerability detection, we adopt

Table 1. Function-level and statement-level statistics of REVEAL, D2A, and Big-Vul datasets, where Ratio denotes the ratio of vulnerable to non-vulnerable functions and Stat abbreviates Statements

Dataset	Vulnerable	Benign	Total	Ratio	Vulnerable stat	Benign stat	Total stat
REVEAL	2240	20 494	22 734	1:9.15	-	-	-
D2A	17 026	33 403	50 429	1:1.96	55 282	1244 333	1299 615
Big-Vul	10 900	177 736	188 636	1:16.31	46 513	689 412	735 925

four widely used metrics to evaluate function-level detection performance: accuracy, precision, recall, and F1-score. These metrics provide a comprehensive assessment of the model's ability to correctly identify vulnerable functions.

Statement-level vulnerability localization. To evaluate the accuracy of statement-level vulnerability localization, we follow the approach of LineVul [20] and employ top-10 accuracy as the primary metric. This metric measures the proportion of vulnerability functions where at least one actual vulnerable statement is correctly identified within the top 10 predicted statements. By narrowing down the search space to the top 10 statements, top-10 accuracy assists security analysts in efficiently identifying the most critical vulnerability-related code segments, making it highly practical for real-world vulnerability audits.

5. EXPERIMENTAL RESULTS

To comprehensively evaluate the effectiveness and practicality of HySELoc, we formulate the following research questions, aiming to validate its technical innovations, generalization capability, and real-world applicability:

RQ1. How well does HySELoc perform in function-level vulnerability detection compared with state-of-the-art methods?

RQ2. How accurately can HySELoc locate vulnerabilities at the line level in comparison with state-of-the-art interpretable baselines?

RQ3. How do code sequences, AST subtrees, and THSG collectively enhance HySELoc's performance through hybrid semantic fusion?

RQ4. Can HySELoc effectively detect vulnerabilities in real-world codebases, including 0-day vulnerabilities?

The motivations behind these research questions aim to evaluate HySELoc's effectiveness in vulnerability detection and localization. We first assess its ability to detect vulnerabilities at the function level, which is critical for large-scale assessments and subsequent localization. Next, we examine its precision in locating line-level vulnerabilities compared with state-of-the-art interpretable baselines. We then explore how combining code sequences, AST subtrees, and THSG enhances performance through hybrid semantic fusion. Finally, we test its effectiveness in detecting vulnerabilities in real-world codebases, including potential 0-day vulnerabilities, to validate its practical applicability.

5.1 Function-level detection performance (RQ1)

Table 2 summarizes the function-level vulnerability detection results of HySELoc and six state-of-the-art baseline methods across three datasets: REVEAL, D2A, and Big-Vul. The evaluation metrics include accuracy, precision, recall, and F1-score.

The experimental results consistently demonstrate the superior performance of HySELoc, achieving the highest F1-score

across all datasets. Specifically, it outperforms the best baselines by 0.97, 3.46, and 2.32 percentage points on REVEAL, D2A, and Big-Vul, respectively. These consistent gains highlight HySELoc's reliable performance across datasets. Among 12 evaluation cases, HySELoc outperforms all baselines in 10 cases, highlighting its stability and effectiveness in function-level vulnerability detection.

The performance trends across models reflect their inherent architectural characteristics. Sequence-based methods such as VulDeePecker and SySeVR, which rely solely on token sequences, show limited ability to capture structural vulnerability patterns, resulting in low F1-scores on REVEAL (16.37% and 29.40%, respectively). Graph-based approaches like REVEAL and IVDetect achieve moderate improvements (41.88% and 45.04% on REVEAL) by incorporating semantic analysis, but their performance remains constrained due to coarse-grained node representations and insufficient contextual modeling. Transformer-based LineVul benefits from self-attention to capture token-level context and achieves a high F1-score on Big-Vul (91.42%), yet it struggles on more structurally complex datasets such as REVEAL (50.87%), indicating limited generalization. The hybrid model VELVET, which integrates GNNs and Transformers, performs competitively across datasets (F1-scores of 51.92%, 81.36%, and 86.87% on REVEAL, D2A, and Big-Vul, respectively), but does not significantly outperform LineVul. This may be attributed to VELVET's focus on statement-level vulnerability localization, which limits its effectiveness in function-level classification tasks.

In contrast, HySELoc consistently performs better across all datasets. Its advantage lies in the integration of code sequences, AST subtrees, and THSG, which together capture syntax, context, and token-level semantics. This multilevel fusion enables HySELoc to adapt to diverse code structures and dataset characteristics, effectively addressing the limitations of models based on single or coarse representations.

5.2 Statement-level localization precision (RQ2)

In Fig. 3, we present the experimental results of HySELoc and five baseline methods on two datasets based on top-10 accuracy. It is crucial to emphasize that, except for LineVul, the other four model-agnostic interpretable methods compute contributions from three sub-models first and then employ the same weighted fusion strategy as HySELoc to obtain the final results.

The experiments demonstrate that on the Big-Vul dataset, HySELoc achieves a top-10 accuracy of 81%, significantly outperforming the second-best methods Saliency and DeepLiftSHAP (both at 70%). On the D2A dataset, HySELoc achieves a top-10 accuracy of 90%, which is 13 percentage points higher than the second-best method LIG (77%). Notably, Saliency, DeepLiftSHAP, and LIG are all model-agnostic methods, indicating that HySELoc excels over model-agnostic approaches in vulnerability localization tasks. Its superiority primarily stems from direct utilization of internal model features, enabling precise identification of vulnerability patterns. In contrast, model-agnostic methods

Table 2. Comparison of function-level vulnerability detection performance between HySELoc and baseline methods across three datasets, with the best-performing results for each metric highlighted in bold

Baselines\Metrics(%)	REVEAL				D2A				Big-Vul			
	Acc	Pre	Rec	F1	Acc	Pre	Rec	F1	Acc	Pre	Rec	F1
VulDeePecker	76.52	22.31	12.93	16.37	49.85	48.31	59.92	53.49	84.46	37.06	13.34	19.62
SySeVR	77.53	42.27	22.54	29.4	57.38	52.87	71.03	60.62	90.42	32.24	14.47	19.97
REVEAL	84.24	31.26	63.43	41.88	69.83	53.46	75.59	62.63	87.14	17.22	34.04	22.87
IVDetect	88.92	39.51	52.36	45.04	78.64	77.37	80.06	78.69	93.84	23.34	72.47	35.31
VELVET	89.92	50.43	53.51	51.92	82.26	81.17	81.56	81.36	98.74	89.63	84.27	86.87
LineVul	88.87	47.99	54.13	50.87	88.02	80.25	85.79	82.93	99.09	97.12	86.35	91.42
HySELoc	91.28	50.26	55.82	52.89	91.43	84.54	88.32	86.39	99.23	98.33	89.56	93.74

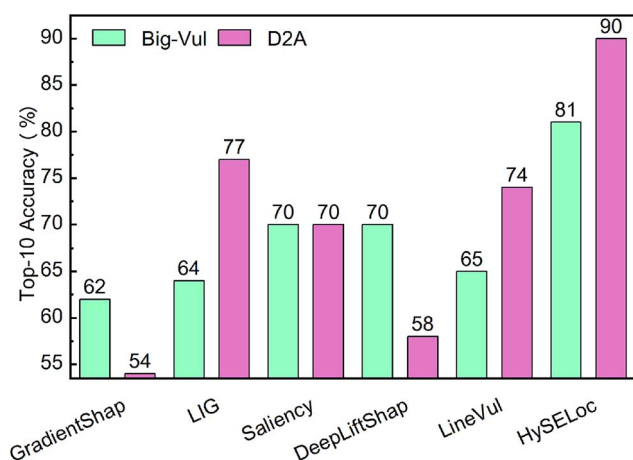


Figure 3. Results of the performance comparison between HySELoc and the baseline approach for statement-level vulnerability localization on the two datasets.

rely on input perturbations to approximate feature importance, resulting in insufficient feature correlations and thereby limiting localization performance.

Importantly, LineVul significantly underperforms compared with the weighted aggregation methods across all performance metrics. Specifically, on both the Big-Vul and D2A datasets, LineVul fails to reach the second-best level in key performance indicators. For instance, its top-10 accuracy on the Big-Vul dataset is only 65%, and on D2A it reaches just 74%, both significantly lower than the second-best performance achieved by model-agnostic methods using weighted fusion strategies. This substantial performance gap highlights LineVul's limitations in interpretable vulnerability localization tasks. The main issue lies in its reliance solely on attention mechanisms for code token sequences, which overlooks structural information and dependency relationships within the code. As a result, LineVul struggles to accurately identify vulnerability patterns. In contrast, other methods capture vulnerability features more effectively by integrating subtree sequences and THSG. Subtree sequences preserve the hierarchical structure of the code, enhancing the modeling of critical syntactic features, while THSG strengthens the capture of complex vulnerability patterns through semantic dependencies. The weighted aggregation strategy used in HySELoc further integrates multilevel information, allowing it to better address diverse code vulnerability patterns and achieve superior localization accuracy.

Beyond accuracy, HySELoc also improves interpretability by aggregating attention-based contribution scores from three

Table 3. Results of comparison of F1-scores(%) for function-level vulnerability detection on three datasets for different fusion strategies

Combinations	REVEAL	D2A	Big-Vul
Seq	50.87	82.93	91.27
SubtreesSeq	40.28	83.3	37.83
THSG	37.18	70.33	75.42
Seq+SubtreesSeq	51.1	83.98	91.49
SubtreesSeq+THSG	43.71	85.75	77.09
Seq+THSG	52.64	85.53	92.79
HySELoc	52.89	86.39	93.74

structurally complementary views. This enables fine-grained, semantically meaningful explanations aligned with program structure. To qualitatively evaluate this capability, we present a case study in Section 6.3, where attribution results from different methods are visually compared and manually assessed by experienced researchers.

5.3 Hybrid semantic-syntactic analysis (RQ3)

(1) Effects of hybrid semantic fusion in HySELoc

To investigate the synergistic effects of multilevel code representations, we systematically evaluated the impact of different fusion strategies on vulnerability detection and localization through ablation experiments. Table 3 presents the F1-scores for vulnerability identification across three datasets using various fusion methods, while Fig. 4 illustrates the top-10 accuracy results for vulnerability localization on two datasets.

Table 3 shows that the model integrating all three representation types achieves the highest F1-scores across all datasets, outperforming individual or pairwise combinations. While certain single representations excel on specific datasets, such as code sequences on REVEAL (50.87%) and Big-Vul (91.27%), and AST subtree sequences on D2A (83.3%), their stand-alone performance is inconsistent, highlighting the need for multilevel fusion to address diverse vulnerability patterns. Pairwise combinations reveal complementary relationships. For example, Seq+THSG performs nearly optimally on REVEAL (52.64%) and Big-Vul (92.79%), indicating synergy between sequential context and semantic dependencies. Similarly, SubtreesSeq+THSG excels on D2A (85.75%), suggesting that combining structured syntax with semantic graphs enhances detection. In contrast, Seq+SubtreesSeq underperforms across all datasets, emphasizing the limitations of relying solely on syntactic hierarchies without fine-grained semantic information.

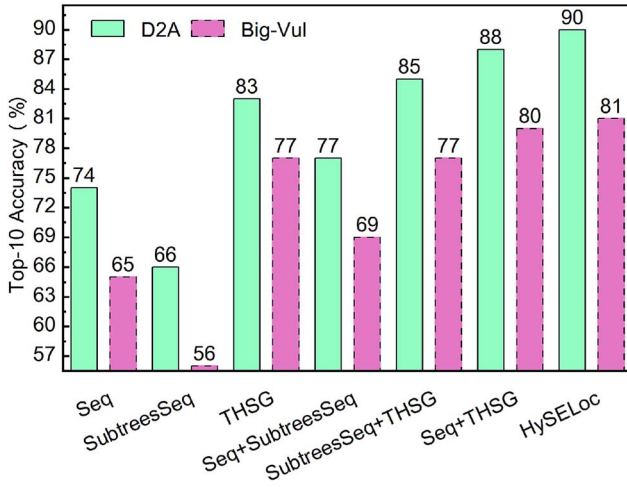


Figure 4. Results of the performance comparison of different fusion strategies for statement-level vulnerability localization on two datasets.

Table 4. Top-10 accuracy (%) on D2A and Big-Vul using different semantic edge configurations in THSG, where Group1 retains only TokenFlow edges, Group2 removes ControlFlow, Group3 removes ComesFrom and ComputedFrom, Group4 removes LastWrite and LastUse, and Group5 includes all six edge types (i.e. the complete THSG used in HySELoc)

Edge group	D2A	Big-Vul
Group1 (TokenFlow only)	77	70
Group2 (-ControlFlow)	86	78
Group3 (-ComesFrom, -ComputedFrom)	82	74
Group4 (-LastWrite, -LastUse)	84	77
Group5 (Full THSG, HySELoc)	90	81

Figure 4 further validates the importance of multilevel fusion in vulnerability localization, with THSG playing a critical role in improving accuracy. In the top-10 accuracy evaluation, THSG alone outperforms both code sequences and subtree sequences, achieving 83% on D2A and 77% on Big-Vul, demonstrating its ability to capture fine-grained semantic information. However, single representations still have limitations, and their fusion enhances localization performance. For instance, Seq+THSG improves top-10 accuracy to 88% on D2A and 80% on Big-Vul, showing that integrating sequential information with semantic dependencies boosts precision.

In summary, HySELoc enhances vulnerability detection and localization by fusing multilevel code representations, including code sequences, AST subtree sequences, and THSG, to comprehensively capture code features. This integration addresses the limitations of single representations and significantly improves detection accuracy and localization precision.

(2) Effect of semantic edge types on interpretability

As illustrated in Fig. 4, THSG plays a critical role in vulnerability localization. To further analyze the impact of different semantic edge types in THSG on localization performance and interpretability, we conducted an edge-type ablation study within the full HySELoc model. The results are shown in Table 4.

The full THSG configuration (Group5) achieves the highest top-10 accuracy, reaching 90% on D2A and 81% on Big-Vul, confirming the effectiveness of integrating diverse semantic relations. In contrast, using only TokenFlow edges (Group1) leads to a substantial performance drop, indicating that lexical connections

alone are insufficient for capturing essential semantic relationships in code. Removing ControlFlow edges (Group2) results in accuracy drops of 4% on D2A and 3% on Big-Vul, suggesting that control dependencies are critical for identifying execution-sensitive vulnerabilities. Similarly, omitting value-propagation edges (ComesFrom, ComputedFrom, Group3) or memory-tracking edges (LastWrite, LastUse, Group4) degrades performance, underscoring the role of data and state flow in modeling complex vulnerability patterns. These results demonstrate that all semantic edge types make distinct and complementary contributions to accurate and robust vulnerability localization.

From an interpretability perspective, semantic edges such as value-flow and memory-access encourage the model to attend to code regions that are often relevant to vulnerability causes or fix locations. Their inclusion helps HySELoc produce attribution results that tend to align more closely with human reasoning patterns, thereby improving practical explainability in real-world auditing tasks.

5.4 Real-world applicability (RQ4)

To evaluate the real-world applicability of HySELoc, we conducted experiments on five widely used open-source projects: Linux Kernel, Redis, FFmpeg, Gpac, and OpenSSL. For each project, we selected a specific historical version and collected relevant CVE records from the CVE database [47]. We then manually identified vulnerable functions by referring to the associated vulnerability reports and patches.

Prior work [48] has pointed out that benchmark datasets such as REVEAL, D2A, and Big-Vul often suffer from function-level label ambiguity due to their reliance on vulnerability patches that may span multiple functions. To mitigate this concern, we ensured that each CVE instance used in our evaluation corresponds to a clearly defined vulnerability confined to a single function. This reduces annotation noise and provides a more controlled basis for model evaluation at the function level.

To enable a fair comparison, we include LineVul and VELVET as baselines due to their strong performance in vulnerability detection tasks. Table 5 presents the detection results of all three methods on the five selected projects. HySELoc achieves detection rates ranging from 40% to 100%, with an overall average of 47.12%. In contrast, LineVul and VELVET achieve average rates of 41.15% and 41.94%, respectively. These results confirm that HySELoc consistently outperforms both baselines in realistic, non-synthetic environments. Notably, on the Linux Kernel (v5.15.0), which contains 997 known vulnerabilities, HySELoc correctly detects 466 cases (46.74%), demonstrating its scalability and robustness in large codebases.

Table 6 lists representative vulnerabilities detected by HySELoc in open-source projects. Among them, the two CVEs highlighted in bold are genuine zero-day vulnerabilities identified through real-world scanning. To validate HySELoc's capability in uncovering unknown vulnerabilities, we extracted all C functions from the 2.5-DEV version of the Gpac project and performed a comprehensive scan. The flagged functions were manually reviewed, and the confirmed samples were further validated using dynamic analysis. Notably, these two zero-day vulnerabilities were detected only by HySELoc, while LineVul and VELVET failed to identify them. We reported the validated issues to the developers, who subsequently assigned them identifiers **CVE-2024-6062** [49] and **CVE-2024-6064** [50]. This result demonstrates HySELoc's effectiveness in identifying previously unknown security threats in practical, real-world applications.

Table 5. Detection performance of different methods on real-world open-source projects, where “Detected” denotes the number of CVEs identified and “Rate (%)” is the ratio of detected to total CVEs

Project	Version	CVE total	LineVul		VELVET		HySELoc	
			Detected	Rate (%)	Detected	Rate (%)	Detected	Rate (%)
Linux kernel	V5.15.0	997	412	41.32	419	42.03	466	46.74
Redis	V7.0 rc1	4	2	50.00	2	50.00	3	75.00
FFmpeg	V5.10	4	3	75.00	3	75.00	4	100.0
Gpac	V2.2.0	25	6	24.00	6	24.00	10	40.00
Openssl	V3.0.4	12	6	50.00	7	58.33	8	66.67
Total		1042	429	41.17	437	41.94	491	47.12

Table 6. Examples of vulnerabilities detected by HySELoc in open-source projects, with the two CVEs highlighted in bold representing 0-day vulnerabilities detected by HySELoc

Project	CVE ID	CWE type	Vulnerability file	Release date	Version
Linux Kernel	CVE-2024-26883	CWE-119	drivers/net/usb/hso.c	2024-04-17	V5.15.0
	CVE-2024-26585	CWE-362	net/tls/tls_sw.c	2024-02-21	V5.15.0
Redis	CVE-2023-41053	CWE-269	src/db.c	2023-09-06	V7.0 rc1
	CVE-2023-28856	CWE-20	src/t_hash.c	2023-04-18	V7.0 rc1
FFmpeg	CVE-2024-22862	CWE-190	libavcodec/jpegxl_parser.c	2024-01-27	V5.10
	CVE-2024-22860	CWE-190	libavformat/jpegxl_anim_dec.c	2024-01-27	V5.10
Gpac	CVE-2023-28838	CWE-125	src/filter_core/filter_session.c	2023-05-22	V2.2.0
	CVE-2024-6062	CWE-476	filters/load_text.c	2024-06-17	2.5-DEV
	CVE-2024-6064	CWE-416	scene_manager/loader_xmt.c	2024-06-17	2.5-DEV
Openssl	CVE-2023-0215	CWE-416	crypto/asn1/bio_ndef.c	2023-02-08	V3.0.4
	CVE-2022-2274	CWE-787	crypto/bn/rsaz_exp_x2.c	2022-07-01	V3.0.4

6. DISCUSSION

6.1 Threats to validity

Internal validity. A potential threat stems from label noise in benchmark datasets such as REVEAL, D2A, and Big-Vul, which are based on vulnerability-fix commits that may span multiple functions. To mitigate this, we conducted real-world evaluations on five open-source projects, with each CVE instance manually verified to be confined to a single function. This reduces annotation ambiguity and provides a more reliable evaluation setting. In these scenarios, HySELoc consistently outperformed strong baselines. Another threat lies in relying on attention-based attribution for line-level localization, which may not always align with true causality. HySELoc addresses this by integrating attention signals from code sequences, AST subtrees, and semantic graphs to enhance robustness. Additionally, although we did not explicitly analyze performance across function lengths or vulnerability types due to limited fine-grained labels, our real-world evaluations cover diverse code structures and show consistent performance, suggesting general robustness. More systematic analysis is left for future work.

External validity. Our evaluation is conducted on C/C++ open-source projects with publicly disclosed CVEs. While these projects are widely used, they may not reflect the characteristics of proprietary or domain-specific systems. Moreover, HySELoc has not been tested on other languages or programming paradigms, and its effectiveness on non-code-based or logic-level vulnerabilities remains unverified. These factors may limit the generalizability of our results in broader real-world contexts.

6.2 Limitations and future work

While HySELoc demonstrates strong performance in both detection and localization tasks, several limitations remain to be addressed.

First, the incorporation of THSG and multi-model fusion introduces additional computational overhead in preprocessing and training. Although THSG is more lightweight than traditional representations such as CPG and PDG, its construction and integration with Transformer and GCN modules inevitably increase time and memory consumption. Future work will explore more efficient representation fusion and model compression strategies to improve scalability.

Second, the applicability of HySELoc to large-scale industrial codebases and CI/CD pipelines has not been extensively evaluated. Since real-world systems involve massive and evolving code, we plan to investigate incremental analysis and deployment-friendly variants of HySELoc for integration into DevSecOps workflows.

Third, potential failure cases may occur when AST parsing fails or semantic edges in THSG are sparse or noisy, especially in incomplete or obfuscated code. Although Tree-sitter achieved a 100% parsing rate in our experiments, robustness under broader conditions remains an open challenge.

Finally, HySELoc currently supports only C/C++ and focuses on generic vulnerabilities; its generalizability to other programming languages and vulnerability types is yet to be validated. To address this, we aim to extend support for additional languages and incorporate multi-type vulnerability classification via multi-task learning. Moreover, as a static analysis approach, HySELoc lacks runtime information, which may limit its effectiveness in detecting execution-sensitive vulnerabilities. Future work will consider hybrid techniques that combine static structure with dynamic traces to improve accuracy and robustness.

6.3 Case study

To validate the practical interpretability of HySELoc, we present two real-world cases that compare its localization results with

CVE-2023-52637	Localization Results
<pre> 1. static bool j1939_sk_match_filter(struct j1939_sock *jsk, 2. const struct j1939_sk_buff_cb *skcb) 3. { 4. const struct j1939_filter *f = jsk->filters; 5. int nfilter = jsk->nfilters; 6. if (!nfilter) 7. return true; 8. for (; nfilter; ++f, --nfilter) { 9. if ((skcb->addr.pgn & f->pgn_mask) != f->pgn) 10. continue; 11. if ((skcb->addr.sa & f->addr_mask) != f->addr) 12. continue; 13. if ((skcb->addr.src_name & f->name_mask) != f->name) 14. continue; 15. return true; 16. } 17. return false; 18. }</pre>	IVDetect: 6, 8, 9, 10, 11, 12, 13, 14, 15, 16 GradientSHAP: 13, 11, 1, 2, 9 LIG: 11, 13, 1, 2, 4 Saliency: 11, 13, 1, 2, 4 DeepLiftSHAP: 13, 11, 1, 2, 8 LineVul: 13, 11, 9, 1, 2 HySELoc(Ours): 5, 4, 8, 7, 15

(a) CVE-2023-52637

CVE-2024-36949	Localization Results
<pre> 1. void kgd2kfd_suspend(struct kfd_dev *kfd, bool run_pm) 2. { 3. struct kfd_node *node; 4. int i; 5. int count; 6. if (!kfd->init_complete) 7. return; 8. if (!run_pm) { 9. mutex_lock(&kfd->processes_mutex); 10. count = ++kfd->locked; 11. mutex_unlock(&kfd->processes_mutex); 12. if (count == 1) 13. kfd_suspend_all_processes(); 14. } 15. for (i = 0; i < kfd->num_nodes; i++) { 16. node = kfd->nodes[i]; 17. node->dqm->ops.stop(node->dqm); 18. } 19. }</pre>	IVDetect: 6, 15, 16, 17, 18 GradientSHAP: 1, 17, 15, 9, 11 LIG: 1, 15, 17, 9, 11 Saliency: 1, 17, 9, 11, 13 DeepLiftSHAP: 1, 15, 17, 16, 9 LineVul: 1, 17, 15, 11, 16 HySELoc(Ours): 11, 10, 12, 5, 1

(b) CVE-2024-36949

Figure 5. Case study of vulnerability localization, where red highlights indicate ground-truth vulnerable statements, the red font in the result columns marks lines identified as vulnerable by each method, IVDetect outputs all predicted vulnerable lines, and for other methods only the top-5 lines ranked by predicted vulnerability scores are shown.

representative model-agnostic and model-specific interpretability methods, as illustrated in Fig. 5.

The first case, CVE-2023-52637, is a use-after-free vulnerability in the Linux kernel, caused by unsafe access to `jsk->filters`. The second case, CVE-2024-36949, involves a synchronization flaw in the AMD GPU driver, where improper coordination between devices during reset may lead to page faults. In both cases, HySELoc accurately identifies all four ground-truth vulnerable lines within its top-5 predictions. In contrast, IVDetect, which utilizes GNNExplainer, highlights large spans of lines (e.g. lines 8–16 or 15–18), resulting in coarse-grained and less informative explanations. Gradient-based model-agnostic methods (such as LIG, Saliency, and DeepLiftSHAP) each include at most one correct line per case, with the remaining predictions scattered across irrelevant code. Similarly, LineVul detects only

one true positive in the second case and fails entirely in the first.

These results demonstrate that HySELoc offers more precise and focused localization, with explanations that are semantically and structurally meaningful. The highlighted lines produced by HySELoc align closely with actual vulnerability locations and fix patterns, thereby making its outputs more interpretable and actionable for human analysts.

7. CONCLUSION

In this study, we propose HySELoc, a hybrid semantic enhancement framework for explainable vulnerability localization. By integrating code sequences, AST subtree sequences, and THSG, HySELoc effectively captures hierarchical syntactic features and

fine-grained semantic dependencies, enabling precise line-level vulnerability detection. Experimental results demonstrate that HySELoc significantly outperforms state-of-the-art baselines in both F1-scores and top-10 accuracy, achieving 90% on the D2A dataset and 81% on the Big-Vul dataset. Beyond benchmark evaluations, HySELoc successfully identified two zero-day vulnerabilities (CVE-2024-6062/6064) in the Gpac project and detected historical vulnerabilities in large-scale open-source projects such as the Linux kernel, Redis, and OpenSSL, showcasing its practical utility.

FUNDING

This work was supported in part by the National Natural Science Foundation of China under Grant U24B20147 and Grant U2133208; and in part by the Major Science and Technology Special Project of Sichuan Province under Grant 2024ZYD0269, 2024ZHCG0195, and 2024ZDZX0044.

DATA AVAILABILITY

The data underlying this article will be shared on reasonable request to the corresponding author.

DISCLOSURES

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

REFERENCES

- Kaur A, Nayyar R. A comparative study of static code analysis tools for vulnerability detection in C/C++ and java source code. *Procedia Comput Sci* 2020;**171**:2023–9. <https://doi.org/10.1016/j.procs.2020.04.217>.
- Zio E. Challenges in the vulnerability and risk analysis of critical infrastructures. *Reliab Eng Syst Saf* 2016;**152**:137–50. <https://doi.org/10.1016/j.res.2016.02.009>.
- National Vulnerability Database. CVE-2021-44228 Detail 2021.
- Qualys. Infographic: Log4shell vulnerability impact by the numbers. 2022.
- National Vulnerability Database. CVE-2014-0160 Detail 2014.
- Pew Research Center. Heartbleed's impact. Pew Research Center, 2014. Available at: <https://www.pewresearch.org/internet/2014/04/30/heartbleeds-impact/>. Accessed 27 Aug. 2025.
- Google Code Archive. Rough auditing tool for security (RATS). 2024.
- Wheeler DA. Flawfinder <https://dwheeler.com/flawfinder/>. 2024.
- Cppcheck Developers. Cppcheck—a tool for static c/c++ code analysis <https://cppcheck.sourceforge.io/>. 2024.
- Checkmarx. Application security testing tool—checkmarx appsec solution <https://checkmarx.com/>. 2024.
- Ding Y, Fu Y, Ibrahim O. et al. Vulnerability detection with code language models: how far are we? 2024; *arXiv preprint arXiv:2403.18624*.
- Tang W, Tang M, Ban M. et al. CSGVD: a deep learning approach combining sequence and graph embedding for source code vulnerability detection. *J Syst Softw* 2023;**199**:111623. <https://doi.org/10.1016/j.jss.2023.111623>.
- Cao S, Sun X, Wu X. et al. Coca: improving and explaining graph neural network-based vulnerability detection systems. In: *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering (ICSE '24)*. Lisbon, Portugal, 14–20 April. New York, NY, USA: IEEE Computer Society, 2024, 1911–23. <https://doi.org/10.1145/3597503.3639168>.
- Wen X-C, Chen Y, Gao C. et al. Vulnerability detection with graph simplification and enhanced graph representation learning. In: *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE '23)*, Melbourne, Australia, 14–20 May. Piscataway, NJ, USA: IEEE/Association for Computing Machinery, 2023, 2275–86. <https://doi.org/10.1109/ICSE48619.2023.00191>.
- Wu Y, Zou D, Dou S. et al. VulCNN: An image-inspired scalable vulnerability detection system. In: *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*, Pittsburgh, PA, USA, 21–29 May 2022. New York, NY, USA: ACM, 2022. <https://doi.org/10.1145/3510003.3510229>.
- Cao S, Sun X, Bo L. et al. BGNN4VD: constructing bidirectional graph neural-network for vulnerability detection. *Inf Software Technol* 2021;**136**:106576. <https://doi.org/10.1016/j.infsof.2021.106576>.
- Ding Y, Suneja S, Zheng Y. et al. VELVET: a novel ensemble learning approach to automatically locate vulnerable statements. In: *Proceedings of the 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER '22)*, Los Alamitos, CA, USA: IEEE Computer Society, 2022, 959–70. <https://doi.org/10.1109/SANER53432.2022.00114>.
- Hin D, Kan A, Chen H. et al. LineVD: statement-level vulnerability detection using graph neural networks. In: *Proceedings of the 19th International Conference on Mining Software Repositories (MSR '22)*, Pittsburgh, PA, USA. New York, NY, USA: Association for Computing Machinery, 2022, 596–607. <https://doi.org/10.1145/3524842.3527949>.
- Li Z, Zou D, Shouhuai X. et al. VulDeeLocator: a deep learning-based fine-grained vulnerability detector. *IEEE Trans Dependable Secure Comput* 2021;**19**:2821–37.
- Fu M, Tantithamthavorn C. LineVul: a Transformer-based line-level vulnerability prediction. In: *Proceedings of the 2022 IEEE/ACM 19th International Conference on Mining Software Repositories (MSR '22)*, Pittsburgh, PA, USA. New York, NY, USA: ACM, 2022, 13 pages. <https://doi.org/10.1145/3524842.3528452>.
- Li Z, Zou D, Xu S. et al. VulDeePecker: a deep learning-based system for vulnerability detection. In: *Proceedings of the Network and Distributed Systems Security Symposium (NDSS 2018)*, San Diego, CA, USA, 18–21 February. Reston, VA, USA: Internet Society, 2018. <https://doi.org/10.14722/ndss.2018.23158>.
- Li Z, Zou D, Shouhuai X. et al. SySeVR: a framework for using deep learning to detect software vulnerabilities. *IEEE Trans Dependable Secure Comput* 2021;**19**:2244–58.
- Feng Z, Guo D, Tang D. et al. CodeBERT: a pre-trained model for programming and natural languages. In: *Findings of the Association for Computational Linguistics: EMNLP 2020*, Online; Association for Computational Linguistics, 2020, 1536–47. <https://doi.org/10.18653/v1/2020.findings-emnlp.139>.
- Zhang J, Liu Z, Xing H. et al. Vulnerability detection by learning from syntax-based execution paths of code. *IEEE Trans Softw Eng* 2023;**49**:4196–212. <https://doi.org/10.1109/TSE.2023.3286586>.
- Zhou Y, Liu S, Siow J. et al. Devign: effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In: *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*. Red Hook, NY, USA: Curran Associates Inc., 2019, 10197–207.
- Chakraborty S, Krishna R, Ding Y. et al. Deep learning based vulnerability detection: are we there yet? *IEEE Trans Softw Eng* 2021;**48**:3280–96. <https://doi.org/10.1109/TSE.2021.3087402>.

27. Li Y, Wang S, Nguyen TN. Vulnerability detection with fine-grained interpretations. In: *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '21)*, August 23–28, 2021, Athens, Greece. New York, NY, USA: Association for Computing Machinery, 2021, 292–303.
28. Kipf TN, Welling M. Semi-supervised classification with graph convolutional networks. 2017; *arXiv preprint arXiv:1609.02907*.
29. Tian Z, Tian B, Lv J. et al. Enhancing vulnerability detection via ast decomposition and neural sub-tree encoding. *Expert Syst Appl* 2024;**238**:121865. <https://doi.org/10.1016/j.eswa.2023.121865>.
30. Liu R, Wang Y, Haitao X. et al. Vul-LMGNNs: fusing language models and online-distilled graph neural networks for code vulnerability detection. *Inf Fusion* 2025;**115**:102748. <https://doi.org/10.1016/j.inffus.2024.102748>.
31. Lundberg SM, Lee S-I. A unified approach to interpreting model predictions. *Adv Neural Inf Process Syst* 2017;**30**:4765–74.
32. Sundararajan M, Taly A, Yan Q. Axiomatic attribution for deep networks. In: Precup D, Teh YW (eds.), *Proceedings of the 34th International Conference on Machine Learning (ICML '17)*, Vol 70. PMLR, 2017, 3319–28.
33. Simonyan K, Vedaldi A, Zisserman A. Deep inside convolutional networks: visualising image classification models and saliency maps. 2013; *arXiv preprint arXiv:1312.6034*.
34. Ying Z, Bourgeois D, You J. et al. GNNExplainer: generating explanations for graph neural networks. *Advances in neural information processing systems* 2019;**32**:9240–51.
35. Sennrich R, Haddow B, Birch A. Neural machine translation of rare words with subword units. 2015; *arXiv preprint arXiv:1508.07909*.
36. Wang H, Ye G, Tang Z. et al. Combining graph-based learning with automated data collection for code vulnerability detection. *IEEE Trans Inf Forensics Secur* 2020;**16**:1943–58. <https://doi.org/10.1109/TIFS.2020.3044773>.
37. Zhang Y, Yu X, Cui Z. et al. Every document owns its structure: inductive text classification via graph neural networks. In: Jurafsky D, Chai J, Schluter N, Tetreault J (eds.), *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL 2020)*. Online: Association for Computational Linguistics, 2020, 334–39. <https://doi.org/10.18653/v1/2020.acl-main.31>.
38. Islam NT, De La Torre Parra G, Parra DM. et al. An unbiased transformer source code learning with semantic vulnerability graph. In: *2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P '23)*, Delft, Netherlands. Piscataway, NJ, USA: IEEE / Curran Associates, Inc., 2023, 144–59. <https://doi.org/10.1109/EuroSP57164.2023.00018>.
39. Guo D, Ren S, Lu S. et al. GraphCodeBERT: pre-training code representations with data flow. 2020; *arXiv preprint arXiv:2009.08366*.
40. Zheng Y, Pujar S, Lewis B. et al. D2A: a dataset built for AI-based vulnerability detection methods using differential analysis. In: *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP '21)*, Madrid, Spain; Virtual conference, 25–28 May 2021; IEEE, 2021, 111–20.
41. Fan J, Li Y, Wang S. et al. A C/C++ code vulnerability dataset with code changes and CVE summaries. In: Devanbu P, Zimmermann T, Vasilescu B (eds.), *Proceedings of the 17th International Conference on Mining Software Repositories (MSR '20)*, Seoul, Republic of Korea: Association for Computing Machinery, New York, NY, USA; 2020, 508–12.
42. Joern, Open-source code analysis platform for C/C++ based on code property graphs. Available at: <https://joern.io> (Accessed: 17 June 2024).
43. Lin T-Y, Goyal P, Girshick R. et al. Focal loss for dense object detection. In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV '17)*, Venice, Italy: IEEE, 2017, 2980–88.
44. Paszke A, Gross S, Massa F. et al. PyTorch: An imperative style, high-performance deep learning library. In: *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, Vancouver, Canada: Curran Associates, Inc., 2019, 8024–35.
45. Wolf T, Debut L, Sanh V. et al. Transformers: state-of-the-art natural language processing. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, Online: Association for Computational Linguistics, 2020, 38–45.
46. Tree-sitter, Incremental parsing system for programming languages. Available at: <https://tree-sitter.github.io/tree-sitter/> (Accessed: 17 June 2024).
47. CVEDetails, Vulnerability details and information database. Available at: <https://www.cvedetails.com> (Accessed: 17 June 2024).
48. Shestov A, Levichev R, Mussabayev R. et al. Finetuning large language models for vulnerability detection. *IEEE. Access* 2025;**13**: 38889–900. <https://doi.org/10.1109/ACCESS.2025.3546700>.
49. Details CVE. Cve-2024-6062 Detail 2024.
50. Details CVE. Cve-2024-6064 Detail 2024.